

## CSE464 Course Project – Part 3

### Overview

In this project, I improved my graph search program by making the code more organized and easier to understand. I refactored the code and added new features like different search algorithms and a random walk search.

GitHub Link: <https://github.com/speedylight61-del/CSE464-2026-frgibbs.git>

### How to Run the Program

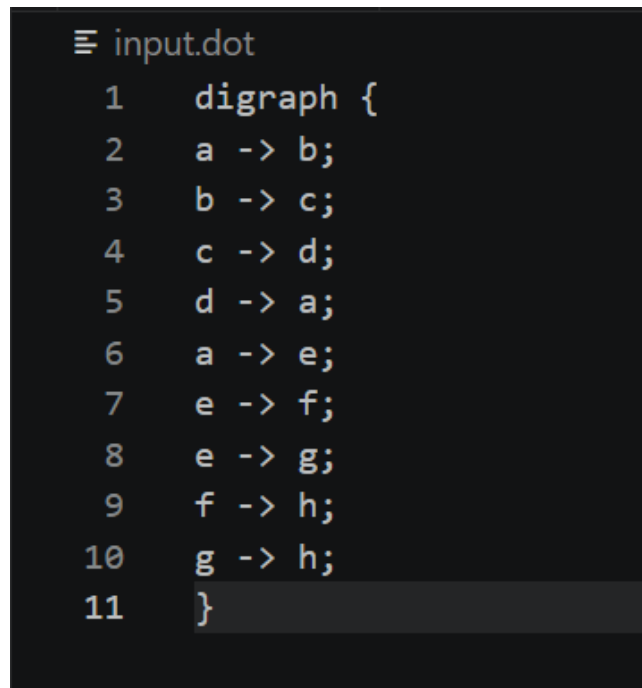
1. Open the project in your IDE (VS Code or IntelliJ)
2. Make sure Maven is installed
3. Open the terminal in the project folder
4. Run this command:

```
mvn exec:java "-Dexec.mainClass=Main"
```

### Input File

The program uses a .dot file as input.

```
digraph {  
a -> b;  
b -> c;  
c -> d;  
d -> a;  
a -> e;  
e -> f;  
e -> g;  
f -> h;  
g -> h;  
}
```

A screenshot of a code editor with a dark background. The file name 'input.dot' is shown in the top left corner. The code is a directed graph definition in DOT format, with line numbers 1 through 11 on the left margin. The code defines a digraph with nodes a, b, c, d, e, f, g, and h, and directed edges between them.

```
1  digraph {  
2    a -> b;  
3    b -> c;  
4    c -> d;  
5    d -> a;  
6    a -> e;  
7    e -> f;  
8    e -> g;  
9    f -> h;  
10   g -> h;  
11   }
```

## 1. Refactoring (Q1)

In this part, I improved my graph search code to make it easier to read and understand. I made several small changes to clean up the code and remove repeated logic.

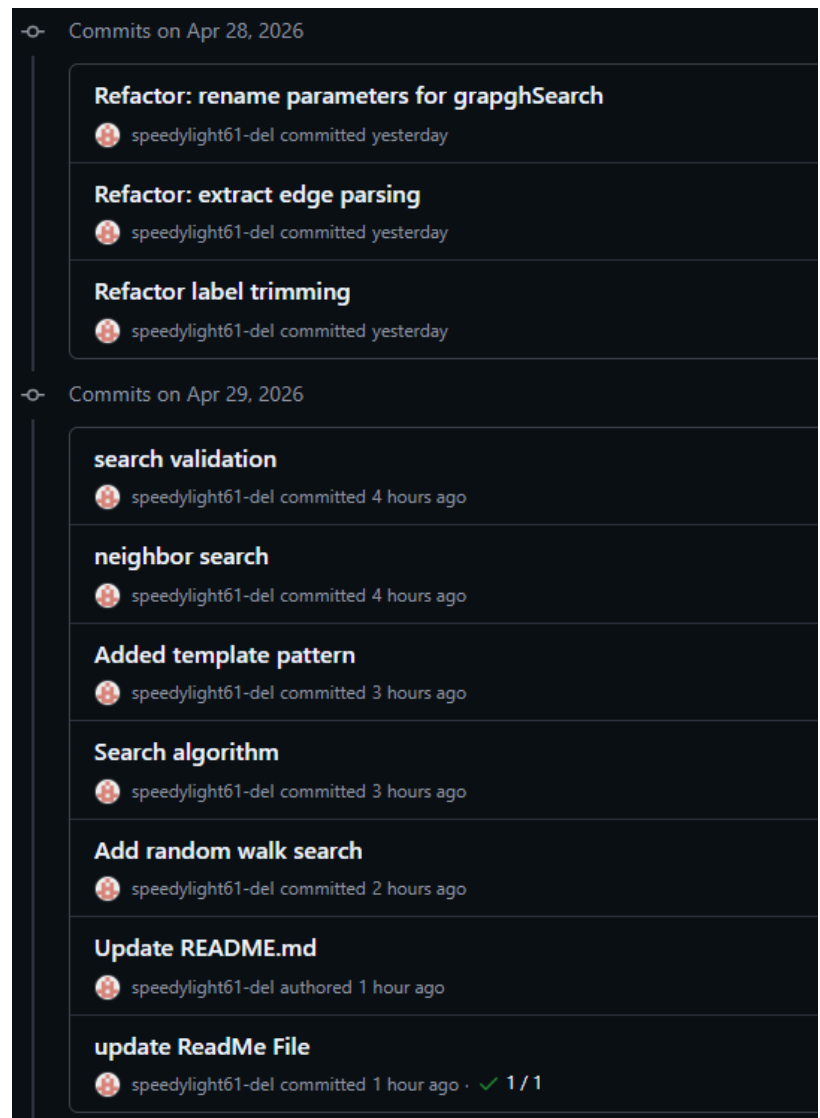
I created a new branch called refactor and made five different improvements to my code. Each change was pushed as its own commit.

Refactoring 1: Renaming parameters

Type: Rename

I changed the names start and end to source and destination so it is easier to understand what the variables mean.

Commit Link:



<https://github.com/speedylight61-del/CSE464-2026-frgibbs/commit/5a43960d18c06d8aa0c9386fd4ca43a6f7edcfbc>

Refactoring 2: Edge parsing helper

Type: Extract method

I noticed I was repeating the same code to split edges, so I made a helper method called splitEdge() to clean it up and avoid repeating code.

Commit link:

<https://github.com/speedylight61-del/CSE464-2026-frgibbs/commit/0d0c8d96c578e69c9d2b624c95760e5bf60d0b4c>

Refactoring 3: Label trimming helper

Type: Extract method

I was using `.trim()` in many places, so I created a helper method called `cleanLabel()` to make the code shorter and cleaner.

Commit link:

<https://github.com/speedylight61-del/CSE464-2026-frgibbs/commit/c57691c8b6dc5087c704034cadeb929eaa7d7aeb>

Refactoring 4: Search validation helper

Type: Extract method

Both BFS and DFS were checking if the nodes were valid, so I moved that logic into one method called `hasValidSearchNodes()`.

Commit link:

<https://github.com/speedylight61-del/CSE464-2026-frgibbs/commit/21481a9da43a03035e4b427360dcad470312500f>

Refactoring 5: Neighbor lookup helper

Type: Extract method

The code for finding neighbors was repeated in BFS and DFS, so I created `getNeighbors()` to make it more organized and easier to read.

Commit link:

<https://github.com/speedylight61-del/CSE464-2026-frgibbs/commit/c1457d45cd1151fb7d6a35ebfde974b4e130530f>

## 2. Template Pattern (Q2)

I used the Template Method Pattern to reduce repeated code between BFS and DFS. I created a base class called `AbstractGraphSearch` that contains the main steps of the search process. This includes checking the nodes, visiting nodes, and building the final path.

Then I created two classes, `BfsSearch` and `DfsSearch`, that extend the base class.

These classes only change how nodes are processed, while the main structure stays the same. This works because both BFS and DFS follow similar steps, but they explore the graph differently. Using this pattern made my code cleaner and avoided repeating the same logic.

```
1 import java.util.*;
2
3 public abstract class AbstractGraphSearch implements Search {
4
5     public String search(Graph graph, String source, String destination) {
6         if (source == null || destination == null) return null;
7         source = source.trim();
8         destination = destination.trim();
9         if (!graph.getNodes().contains(source) || !graph.getNodes().contains(destination)) {
10             return null;
11         }
12         Set<String> visited = new LinkedHashSet<>();
13         Map<String, String> parent = new HashMap<>();
14         setup(source);
15         visited.add(source);
16         while (!isEmpty()) {
17             String current = nextNode();
18             if (current.equals(destination)) {
19                 return buildPath(destination, parent);
20             }
21             for (String neighbor : graph.getNeighbors(current)) {
22                 if (!visited.contains(neighbor)) {
23                     visited.add(neighbor);
24                     parent.put(neighbor, current);
25                     addNode(neighbor);
26                 }
27             }
28         }
29         return null;
30     }
31     protected abstract void setup(String source);
32     protected abstract boolean isEmpty();
33     protected abstract void addNode(String node);
34     protected abstract String nextNode();
35
36     protected String buildPath(String destination, Map<String, String> parent) {
37         String path = destination;
38         String cur = destination;
39         while (parent.containsKey(cur)) {
40             cur = parent.get(cur);
41             path = cur + " -> " + path;
42         }
43         return path;
44     }
45 }
```

Classes that I created:

- AbstractGraphSearch: holds the shared search steps
- BfsSearch: implements BFS using a queue
- DfsSearch: implements DFS using a stack
- Search: interface used to connect the search classes

Commit link:

<https://github.com/speedylight61-del/CSE464-2026-frgibbs/commit/181ce1a356d17e5e7a5357cf106400f1c72a5338>

### 3. Strategy Pattern (Q3)

I used the Strategy Pattern so the program can choose which search algorithm to run based on the algo parameter. Instead of keeping all the BFS and DFS logic directly inside Graph.java, I separated the search behavior into different classes.

I created a Search interface that both BfsSearch and DfsSearch use. In Graph.java, the graphSearch method checks the Algorithm value. If the user chooses BFS, it creates a BfsSearch object. If the user chooses DFS, it creates a DfsSearch object.

This follows the Strategy Pattern because the program can switch between search algorithms without rewriting the main graphSearch method. It also makes the code easier to update later if I need to add another search option.

Classes that I used:

- Search: common interface for the search algorithms
- BfsSearch: handles the BFS search strategy
- DfsSearch: handles the DFS search strategy
- Graph: chooses which strategy to run based on the algo parameter

Commit link:

<https://github.com/speedylight61-del/CSE464-2026-frgibbs/commit/181ce1a356d17e5e7a5357cf106400f1c72a5338>

```
1 public interface Search {  
2     String search(Graph graph, String source, String destination);  
3 }
```

```

public String graphSearch(String source, String destination, Algorithm algo) {
    Search strategy;
    if (algo == Algorithm.BFS) {
        strategy = new BfsSearch();
    } else if (algo == Algorithm.DFS) {
        strategy = new DfsSearch();
    } else {
        strategy = new RandomWalkSearch();
    }
    return strategy.search(this, source, destination);
}

```

#### 4. Random Walk Search (Q4)

I added a new search algorithm called Random Walk. This algorithm starts at the source node and randomly chooses the next node from its neighbors. Because it is random, the path can be different each time the program runs.

I tested the program multiple times and saw different paths in the output. This shows that the algorithm is working correctly. Each time I ran the program, the path changed, like sometimes it went from  $a \rightarrow b \rightarrow c$ , and other times it went from  $a \rightarrow e \rightarrow f \rightarrow h$ . This shows that the algorithm is random and working correctly.

Commit link:

<https://github.com/speedylight61-del/CSE464-2026-frgibbs/commit/4c2ebfc1fb40b7d7aef0ebee168883702849df64>

Graph saved to graphOutput.txt

--- BFS Search ---

Visit Node History: a  
 Visit Node History: a-b  
 Visit Node History: a-e  
 Visit Node History: a-b-c  
 Visit Node History: a-e-f  
 Visit Node History: a-e-g  
 Visit Node History: a-b-c-d  
 Visit Node History: a-e-f-h  
 Found target node: h

--- DFS Search ---

Visit Node History: a  
 Visit Node History: a-e  
 Visit Node History: a-e-g  
 Visit Node History: a-e-g-h  
 Found target node: h

Number of edges: 9  
 Edges:  
 a -> b  
 b -> c  
 c -> d  
 d -> a  
 a -> e  
 e -> f  
 e -> g  
 f -> h  
 g -> h

Graph saved to graphOutput.txt  
 random testing  
 Visit Node History: a  
 Visit Node History: a-b  
 Visit Node History: a-b-c  
 Found target node: c

random testing  
 Visit Node History: a  
 Visit Node History: a-b  
 Visit Node History: a-b-c  
 Found target node: c

random testing  
 Visit Node History: a  
 Visit Node History: a-b  
 Visit Node History: a-b-c  
 Found target node: c

random testing  
 Visit Node History: a  
 Visit Node History: a-b  
 Visit Node History: a-b-c  
 Found target node: c

random testing  
 Visit Node History: a  
 Visit Node History: a-e  
 Visit Node History: a-e-g  
 Visit Node History: a-e-g-h  
 Reached dead end at node h

[INFO] -----  
 [INFO] BUILD SUCCESS

## 5. Pull Request and Code Review (Q5)

I created a pull request on GitHub to show all the changes I made in this project. The pull request includes my refactoring work, the design patterns, and the Random Walk search. I also reviewed my code and added comments to explain what I improved. This helped me make sure my code was clear and correct.

Pull Request Link:

<https://github.com/speedylight61-del/CSE464-2026-frgibbs/pull/1>

I created a pull request for my changes and completed a code review. The pull request was merged after my demo with the TA.

Merge Link: <https://github.com/speedylight61-del/CSE464-2026-frgibbs/pull/1>

